Guía Práctica N.º 11 — Persistencia local offline-first

Estudiante: Julio Fabián Rodríguez Bazán
Aplicación: PharmaMobil (KMP) · Backend: PharmaSoft
Rama: feature/offline-first-rodriguez · Fecha de pruebas: 21/10/2026

1. Resumen

PharmaMobil pasó a ser offline-first: una base SQLite gestionada con SQLDelight es la única fuente de verdad de las pantallas de productos. Los repositorios exponen un Flow que las pantallas observan, y la red queda como un proceso de sincronización en dos sentidos: primero se suben los productos con pendiente = 1 (POST) y después se descarga el catálogo y se reemplazan las filas sincronizadas dentro de una transacción.

2. Implementación

Pieza	Archivo	Función
Esquema	commonMain/sqldelight/.../db/Producto.sq	Tabla producto con columna pendiente y consultas
Drivers	CreadorDeDriver (+Android, +Ios)	AndroidSqliteDriver, NativeSqliteDriver; JdbcSqliteDriver en pruebas
Capa local	data/local/ProductoLocal.kt	observar(): Flow, guardarPendiente (id negativo), reemplazarSincronizados (transacción)
Repositorio	data/repository/ProductoRepositorioOffline.kt	registrar() local + intento de sync; sincronizar(): sube pendientes y luego descarga paginado
Contrato	domain/repository/ProductoRepository.kt	observar(): Flow y sincronizar()
ViewModels	ProductosViewModel, ProductoViewModel	Se suscriben al Flow en init; sin recargas manuales
DI	AppModule, NetworkModule, PlatformModule	PharmaDb, ProductoLocal y driver por plataforma

3. Pruebas automatizadas (commonTest)

Comando: ./gradlew testAndroidHostTest assembleDebug → BUILD SUCCESSFUL. Suite completa del proyecto: 70 pruebas, 0 fallos.

Prueba (ProductoRepositorioOfflineTest)	Verifica	Resultado
loRegistradoSinRedQuedaConIdNegativoYPendiente	Sin red: id < 0 y pendiente = 1; observar() lo emite	OK
sincronizarSubePendientesYReemplazaLaListaLocal	Orden POST → GET; 0 pendientes; lista = servidor	OK
conElServidorFueraDeLineaObservarEmiteLosDatosGuardados	Con 503, observar() entrega la base local sin crash	OK
elViewModelMuestraLoRegistradoSinRecargarAMano	El ViewModel refleja el alta vía Flow	OK

4. Prueba en emulador Android (Pixel 8, API 37.1)

Ciclo completo contra PharmaSoft real (Oracle, Spring Boot en :8080), usuario admin del seed del proyecto.

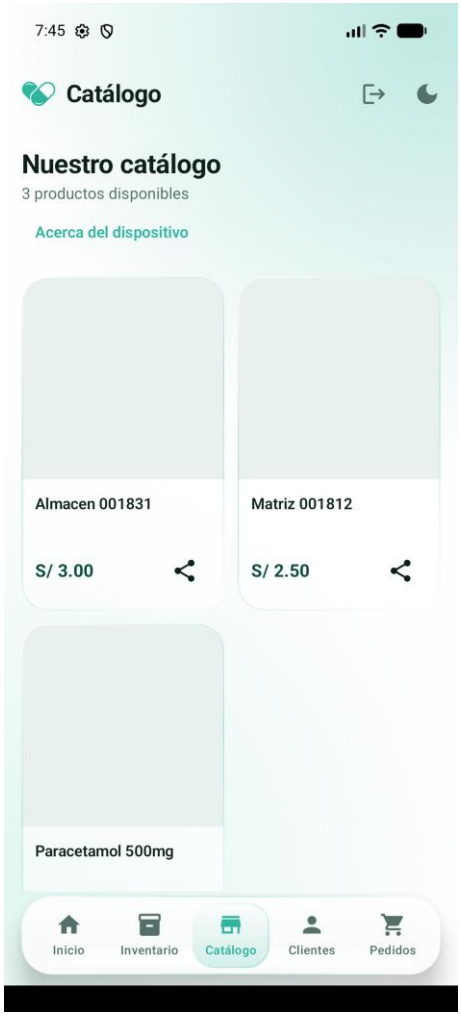


Figura 1. Catálogo con el servidor encendido (3 productos).

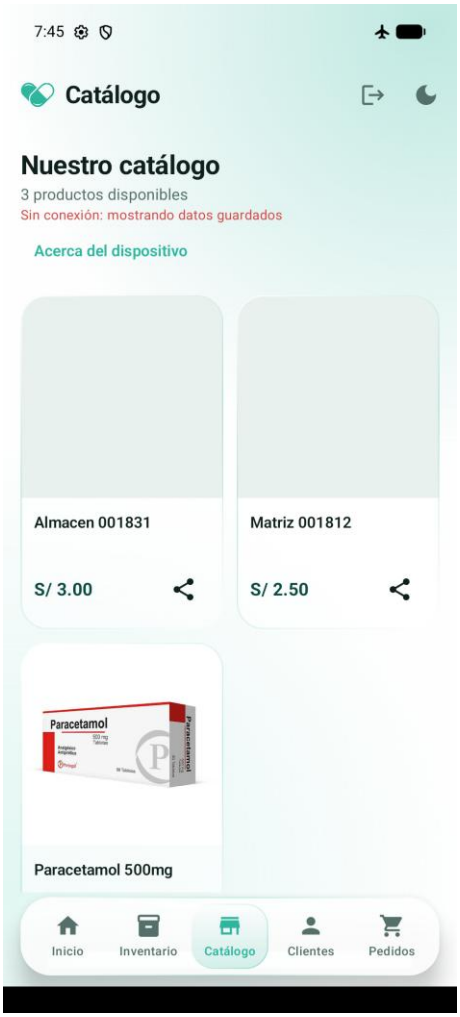


Figura 2. Backend detenido y modo avión: la app abre y lista desde pharma.db, con el aviso «Sin conexión».



Figura 3. Producto «Ibuprofeno_Offline» registrado sin red: aparece de inmediato en el inventario (4 productos).

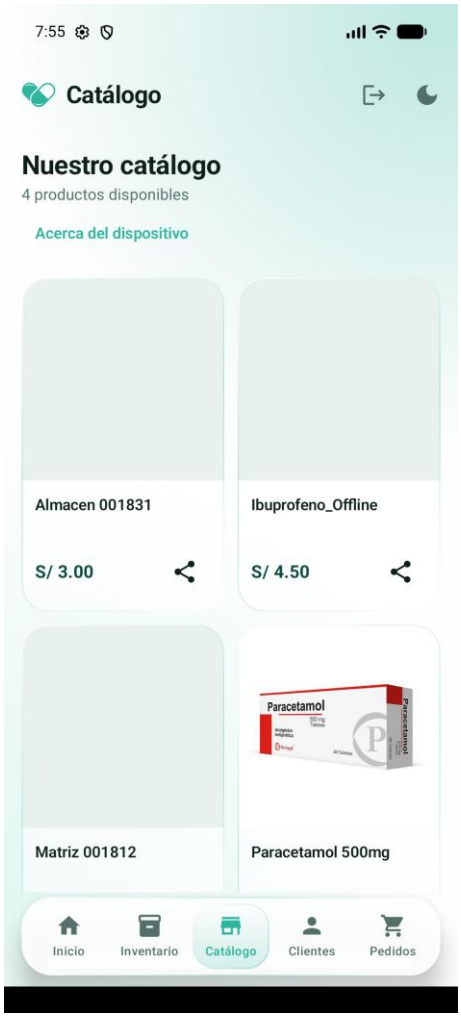


Figura 4. Tras reconectar y reabrir el catálogo: el aviso desapareció y el producto sigue listado.

Estado en PharmaSoft tras la sincronización (GET /api/v1/productos, productos activos):

```
id  nombre  precio  stock  estado
1 Paracetamol 500mg 12.5 40 True
43 Matriz 001812 2.5 10 True
44 Almacen 001831 3 5 True
47 Ibuprofeno_Offline 4.5 25 True
```

El producto creado sin red llegó al servidor con id real 47 (el id local negativo se descartó), sin duplicados.

5. Errores hallados durante la prueba real y corregidos

Hallazgo	Causa	Corrección
sincronizar() fallaba con «El tamaño de página debe estar entre 1 y 100»	Se pedía una página de 1000; el backend limita a 100	Descarga paginada de 100 en 100 hasta la última página
La app se cerraba (NullPointerException) al abrir el Catálogo	productos se declaraba después del init del ViewModel y estaba en null cuando sincronizar() lo usaba	Declarar el estado antes del init

6. Lista de cotejo

N.º	Criterio	Cumple	Evidencia
1	El esquema SQLDelight compila y genera PharmaDb	Sí	Build exitoso
2	Driver en Android, iOS y pruebas	Sí (iOS sin ejecutar)	CreadorDeDriver*, JdbcSqliteDriver
3	La capa local no importa Ktor ni DTO	Sí	ProductoLocal.kt
4	Repositorio expone observar(): Flow y sincronizar()	Sí	ProductoRepository
5	El ViewModel se suscribe al Flow	Sí	Ambos ViewModels; test 4
6	Lista productos con backend apagado	Sí	Figura 2
7	Creado sin red queda pendiente	Sí	Test 1; Figura 3
8	sincronizar() sube antes de bajar	Sí	Test 2; código
9	Aparece en PharmaSoft tras reconectar	Sí	Figura 4; id 47 en el servidor
10	Al menos 3 pruebas en commonTest, todas pasan	Sí	4 pruebas, 70 en total

7. Limitaciones

• iOS: el entorno de desarrollo es Windows; el código de iOS (NativeSqliteDriver y registro en Koin) está implementado pero no se compiló ni ejecutó, por lo que no hay captura en iOS.

• La imagen de un producto creado sin red no puede subirse hasta que el producto se sincronice (la foto se sube con el id real).